

TutorMatch AI - Source Code Documentation

เอกสารรวบรวมซอร์สโค้ดและคำอธิบายโค้ดโครงการอย่างละเอียดทุกไฟล์

ลำดับ	ชื่อไฟล์ซอร์สโค้ด	เทคโนโลยี / หน้าที่หลัก
1	database.py	Python SQLAlchemy (การกำหนด Database Session & Engine)
2	models.py	SQLAlchemy Models (โครงสร้างตารางสมาชิก สแกนหน้า และนัดหมาย)
3	schemas.py	Pydantic Schemas (ตัวแปรคำร้องขอและคำตอบกลับของ API)
4	ai_engine.py	OpenCV Face Recognizer (LBPH) & Thai NLP (TF-IDF Matching)
5	main.py	FastAPI Endpoints (แอปพลิเคชันหลังบ้านเชื่อมต่อหน้าเว็บ)
6	static/index.html	Vanilla HTML5/CSS/JS (สตรีนกเลี้ยงสแกนหน้า และ UI นัดเรียน)

คำอธิบายการจัดทำซอร์สโค้ด:
โครงการนี้ได้รับการออกแบบเป็นสถาปัตยกรรม Web Service
บนฐานการประมวลผลภาษาธรรมชาติภาษาไทย และตัวจดจำใบหน้าแบบออฟไลน์ (Local Biometrics) เพื่อแก้ไขปัญหาความยุ่งยากใน Python 3.14.3
โดยหลีกเลี่ยงการใช้ไลบรารีขนาดใหญ่อย่าง TensorFlow หรือ dlib
และหันมาใช้อัลกอริทึมจดจำใบหน้าดั้งเดิมที่มีความสเถียรสูง (LBPH) ร่วมกับตัวจับภาพ Haar Cascade
ทำให้ซอร์สโค้ดทั้งหมดมีน้ำหนักเบาและประมวลผลได้อย่างรวดเร็วในเครื่องคอมพิวเตอร์ทั่วไป

ไฟล์ที่เขียน: database.py

ตำแหน่งไฟล์ในโปรเจกต์: database.py

รายละเอียดการทำงาน:

ไฟล์นี้ทำหน้าที่กำหนดจุดเชื่อมต่อฐานข้อมูล SQLite (ผ่านไฟล์โลคอล `tutor\_match.db`) และกำหนดคลาส `SessionLocal` เพื่อสร้าง Session ในการอ่านและเขียนข้อมูล รวมถึงฟังก์ชัน `get\_db` ซึ่งเป็น Dependency Injector เพื่อใช้เปิด-ปิดการทำงาน Session อัตโนมัติในแต่ละ API Request

```
from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker

SQLALCHEMY_DATABASE_URL = "sqlite:///./tutor_match.db"

engine = create_engine(
    SQLALCHEMY_DATABASE_URL, connect_args={"check_same_thread": False}
)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

Base = declarative_base()

# Dependency to get db session
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

ไฟล์ที่เขียน: models.py

ตำแหน่งไฟล์ในโปรเจกต์: models.py

รายละเอียดการทำงาน:

ไฟล์นี้ใช้สำหรับประกาศโครงสร้างตารางข้อมูลในระบบผ่านคลาส SQLAlchemy (ORM Models):

- User: ตารางเก็บรายชื่อสมาชิก กำหนดสิทธิ์เป็นนักเรียนหรือติวเตอร์
- BiometricProfile: ตารางเก็บภาพใบหน้าติวเตอร์และนักเรียนที่ลงทะเบียนไว้เป็นรหัส JPEG Base64
- TutorProfile: ตารางเก็บข้อมูลความรู้ความชำนาญ อัตราค่าสอน ช่องทางติดต่อ (LINE, เบอร์โทร, อีเมล) และข้อเสนอแนะนันทมาเรียน
- Appointment: ตารางนัดหมายเรียนพิเศษที่นักเรียนส่งหาติวเตอร์ เก็บวันที่ เวลา โน้ต และสถานะอนุมัติ

```
from sqlalchemy import Column, Integer, String, Float, Text, ForeignKey, DateTime
from sqlalchemy.orm import relationship
from datetime import datetime
from database import Base

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
    username = Column(String, unique=True, index=True, nullable=False)
    name = Column(String, nullable=False)
    role = Column(String, nullable=False) # 'student' or 'tutor'
    line_id = Column(String, unique=True, nullable=True) # LINE ID if integrated
    registered_at = Column(DateTime, default=datetime.utcnow)

    # Relationships
    biometric_profile = relationship("BiometricProfile", uselist=False, back_populates="user", cascade="all, delete-orphan")
    tutor_profile = relationship("TutorProfile", uselist=False, back_populates="user", cascade="all, delete-orphan")

class BiometricProfile(Base):
    __tablename__ = "biometric_profiles"

    id = Column(Integer, primary_key=True, index=True)
    user_id = Column(Integer, ForeignKey("users.id", ondelete="CASCADE"), unique=True)
    face_landmarks = Column(Text, nullable=False) # JSON encoded coordinates of normalized face landmarks
    updated_at = Column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

    user = relationship("User", back_populates="biometric_profile")

class TutorProfile(Base):
    __tablename__ = "tutor_profiles"

    id = Column(Integer, primary_key=True, index=True)
    user_id = Column(Integer, ForeignKey("users.id", ondelete="CASCADE"), unique=True)
    education = Column(String, nullable=False)
    subjects = Column(String, nullable=False) # Comma-separated main subjects
    expertise = Column(Text, nullable=False) # Details of expertise, background
    teaching_style = Column(String, nullable=False) # e.g. 'fun', 'academic', 'friendly'
```

```

price_per_hour = Column(Integer, nullable=False)
availability = Column(String, nullable=False) # e.g. 'weekdays', 'weekends', 'flexible'

# New Contact and Appointment fields
contact_line = Column(String, nullable=True) # LINE ID for contact
contact_phone = Column(String, nullable=True) # Phone number for contact
contact_email = Column(String, nullable=True) # Email for contact
appointment_instruction = Column(Text, nullable=True) # Instructions on how to schedule lessons

profile_vector_text = Column(Text, nullable=True) # Formatted text for search index
updated_at = Column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

user = relationship("User", back_populates="tutor_profile")

class Appointment(Base):
    __tablename__ = "appointments"

    id = Column(Integer, primary_key=True, index=True)
    student_id = Column(Integer, ForeignKey("users.id", ondelete="CASCADE"))
    tutor_id = Column(Integer, ForeignKey("users.id", ondelete="CASCADE"))
    appointment_date = Column(String, nullable=False) # Format: YYYY-MM-DD
    appointment_time = Column(String, nullable=False) # e.g. "13:00 - 15:00"
    notes = Column(Text, nullable=True) # Learning topics or special notes
    status = Column(String, default="pending") # 'pending', 'approved', 'declined'
    created_at = Column(DateTime, default=datetime.utcnow)

    # Relationships to access user data
    student = relationship("User", foreign_keys=[student_id])
    tutor = relationship("User", foreign_keys=[tutor_id])

```

ไฟล์ที่เขียน: schemas.py

ตำแหน่งไฟล์ในโปรเจกต์: schemas.py

รายละเอียดการทำงาน:

ไฟล์กำหนดโครงสร้างพารามิเตอร์ข้อมูล (Data Schema) ด้วย Pydantic Models

ใช้ในการตรวจสอบความถูกต้องและชนิดข้อมูลของข้อมูลขาเข้าและขาออกในระบบ API

เช่น ตรวจสอบความถูกต้องของ Base64 รูปภาพใบหน้า, โครงสร้างการสมัครสมาชิก,

รายละเอียดประวัติติวเตอร์

รวมถึงพารามิเตอร์ที่ใช้ในจองเวลาเรียนและข้อกำหนดในการเรียกคุกกี้หมายเรียน

```
from pydantic import BaseModel
from typing import Optional, List
from datetime import datetime

class UserCreate(BaseModel):
    username: str
    name: str
    role: str # 'student' or 'tutor'

class UserResponse(BaseModel):
    id: int
    username: str
    name: str
    role: str

class Config:
    from_attributes = True

class FaceRegisterRequest(BaseModel):
    username: str
    image_base64: str # Base64 encoded image captured from webcam

class FaceVerifyRequest(BaseModel):
    username: str
    image_base64: str

class TutorProfileCreate(BaseModel):
    education: str
    subjects: str
    expertise: str
    teaching_style: str
    price_per_hour: int
    availability: str
    contact_line: Optional[str] = None
    contact_phone: Optional[str] = None
    contact_email: Optional[str] = None
    appointment_instruction: Optional[str] = None

class TutorProfileResponse(BaseModel):
    id: int
    user_id: int
    education: str
    subjects: str
    expertise: str
    teaching_style: str
    price_per_hour: int
```

```

availability: str
contact_line: Optional[str] = None
contact_phone: Optional[str] = None
contact_email: Optional[str] = None
appointment_instruction: Optional[str] = None

class Config:
    from_attributes = True

class StudentMatchRequest(BaseModel):
    username: str # Verified user
    requirements_text: str
    budget: Optional[int] = None
    subject_filter: Optional[str] = None

class MatchedTutorResponse(BaseModel):
    user_id: int
    name: str
    education: str
    subjects: str
    expertise: str
    teaching_style: str
    price_per_hour: int
    availability: str
    contact_line: Optional[str] = None
    contact_phone: Optional[str] = None
    contact_email: Optional[str] = None
    appointment_instruction: Optional[str] = None
    match_score: float

# Appointment Schemas
class AppointmentCreate(BaseModel):
    student_username: str
    tutor_id: int
    appointment_date: str # YYYY-MM-DD
    appointment_time: str # Slot
    notes: Optional[str] = None

class AppointmentResponse(BaseModel):
    id: int
    student_id: int
    student_name: str
    tutor_id: int
    tutor_name: str
    appointment_date: str
    appointment_time: str
    notes: Optional[str] = None
    status: str
    created_at: datetime

class Config:
    from_attributes = True

```

ไฟล์ที่เขียน: ai_engine.py

ตำแหน่งไฟล์ในโปรเจกต์: ai_engine.py

รายละเอียดการทำงาน:

ไฟล์สมองกล AI ของระบบแบ่งงานออกเป็น 2 อัลกอริทึมหลัก:

1. ระบบวิเคราะห์ภาพสแกนใบหน้า (Face Recognition Engine): ใช้ Haar Cascade ในการตรวจจับขอบตาตำแหน่งใบหน้า กรอปและย่อรูปใบหน้าให้อยู่ในขนาดมาตรฐาน 150x150 พิกเซลแปลงเป็นระดับสีเทา (Grayscale) จากนั้นใช้ ****LBPH Face Recognizer**** ในการเรียนรู้ใบหน้าต้นฉบับและทำนายผลใบหน้าสแกนสดจากกล้อง คำนวณออกมาเป็นคะแนนความต่างระยะห่าง (Distance confidence)
2. ระบบจับคู่ตัวเตอร้อัจฉริยะ (Tutor Matching Engine): ทำการตัดคำภาษาไทยด้วย PyThaiNLP และนำข้อความทั้งหมดมาเข้าสู่อัลกอริทึม ****TF-IDF**** เพื่อแปลงโปรไฟล์ผู้สอนและคำขอของผู้เรียนเป็นเวกเตอร์ จากนั้นเปรียบเทียบด้วยสูตร ****Cosine Similarity**** เพื่อคำนวณคะแนนความเข้ากันได้ (%) และนำเงื่อนไขกรองงบประมาณ/วิชาหลักมาเป็นตัวกรองขั้นต้น

```
import cv2
import numpy as np
import base64
import os
import math
from pythainlp.tokenize import word_tokenize

# Load face cascade
cascade_path = os.path.join(cv2.data.harcascades, "haarcascade_frontalface_default.xml")
face_cascade = cv2.CascadeClassifier(cascade_path)

def extract_face_landmarks(image_bytes: bytes):
    """
    Detects face, crops it, converts to grayscale, resizes to 150x150,
    and returns the base64 string of the cropped face image.
    Returns None if no face is detected.
    """
    nparr = np.frombuffer(image_bytes, np.uint8)
    img = cv2.imdecode(nparr, cv2.IMREAD_COLOR)
    if img is None:
        return None

    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5, minSize=(50, 50))

    if len(faces) == 0:
        return None

    # Get the largest face
    faces = sorted(faces, key=lambda x: x[2] * x[3], reverse=True)
    x, y, w, h = faces[0]

    # Crop and resize
    face_crop = gray[y:y+h, x:x+w]
    face_resized = cv2.resize(face_crop, (150, 150))
```

```

# Encode as JPEG base64
_, buffer = cv2.imencode(&#x27;.jpg', face_resized)
val = base64.b64encode(buffer).decode(&#x27;utf-8')
return val

def verify_faces(registered_face_b64: str, current_image_bytes: bytes, threshold: float = 75.0):
    """
    Compares registered face with the face from the current image.
    Uses LBPH Face Recognizer.
    Returns: (verified: bool, distance: float, error_message: str)
    """
    # Detect and extract current face
    nparr = np.frombuffer(current_image_bytes, np.uint8)
    img = cv2.imdecode(nparr, cv2.IMREAD_COLOR)
    if img is None:
        return False, 999.0, "ไม่สามารถอ่านรูปภาพได้"

    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5, minSize=(50, 50))

    if len(faces) == 0:
        return False, 999.0, "ไม่สามารถตรวจจับใบหน้าในรูปภาพได้ กรุณาลoadภาพหน้าตรงให้ชัดเจน"

    faces = sorted(faces, key=lambda x: x[2] * x[3], reverse=True)
    x, y, w, h = faces[0]
    current_face_resized = cv2.resize(gray[y:y+h, x:x+w], (150, 150))

    # Decode registered face
    try:
        reg_data = base64.b64decode(registered_face_b64)
        reg_nparr = np.frombuffer(reg_data, np.uint8)
        registered_face = cv2.imdecode(reg_nparr, cv2.IMREAD_GRAYSCALE)
    except Exception as e:
        return False, 999.0, f"เกิดข้อผิดพลาดในการโหลดข้อมูลใบหน้าที่ตั้งทะเบียนไว้: {str(e)}"

    if registered_face is None:
        return False, 999.0, "ข้อมูลรูปภาพใบหน้าเดิมมีข้อผิดพลาด"

    # Create LBPH Face Recognizer and train it on the registered face
    try:
        recognizer = cv2.face.LBPHFaceRecognizer_create()
        recognizer.train([registered_face], np.array([1]))
        label, confidence = recognizer.predict(current_face_resized)
    except Exception as e:
        return False, 999.0, f"โมเดลประมวลผลใบหน้าทำงานผิดพลาด: {str(e)}"

    # confidence is the distance. Lower is better.
    # label will always be 1 since we only trained on 1 subject.
    verified = confidence < threshold

    return bool(verified), float(confidence), ""

# --- AI Tutor Matching Engine (NLP based on TF-IDF + Thai Word Tokenization) ---
def compute_tf(tokens):
    tf = {}
    for token in tokens:
        tf[token] = tf.get(token, 0) + 1
    total = len(tokens)

```



```

for token in tf:
    tf[token] = tf[token] / total
return tf

def compute_idf(docs):
    idf = {}
    total_docs = len(docs)
    for doc in docs:
        unique_tokens = set(doc)
        for token in unique_tokens:
            idf[token] = idf.get(token, 0) + 1
    for token in idf:
        idf[token] = math.log(total_docs / idf[token]) + 1
    return idf

def calculate_cosine_similarity(vec1, vec2):
    # Intersection of keys
    keys = set(vec1.keys()) & set(vec2.keys())
    numerator = sum(vec1[k] * vec2[k] for k in keys)

    sum1 = sum(v ** 2 for v in vec1.values())
    sum2 = sum(v ** 2 for v in vec2.values())

    denominator = math.sqrt(sum1) * math.sqrt(sum2)

    if not denominator:
        return 0.0
    return numerator / denominator

def match_tutors(student_req: str, tutors: list, budget: int = None, subject_filter:
str = None):
    """
    Matches student requirements with list of tutors.
    tutors list elements must be dictionaries containing:
    {
        'user_id': int,
        'name': str,
        'subjects': str,
        'expertise': str,
        'teaching_style': str,
        'price_per_hour': int,
        'availability': str,
        'profile_text': str
    }
    Returns: List of tutors with match scores, sorted by score descending.
    """
    if not tutors:
        return []

    # Tokenize student requirement
    student_tokens = [t for t in word_tokenize(student_req.lower()) if t.strip()]

    # Prepare tutor profile documents
    # A tutor document is a combination of subjects + expertise + style
    tutor_docs = []
    for t in tutors:
        # Combine fields into a unified matching text
        text = f'{t["subjects"]} {t["expertise"]} {t["teaching_style"]} {t["availability"]}'.
        lower()
        tutor_tokens = [tok for tok in word_tokenize(text) if tok.strip()]
        tutor_docs.append(tutor_tokens)

```

```

# Build IDF vocabulary using all tutor profiles + student query as background
all_docs = tutor_docs + [student_tokens]
idf = compute_idf(all_docs)

# Calculate TF-IDF vectors
def get_tfidf_vector(tokens):
    tf = compute_tf(tokens)
    vector = {}
    for token, tf_val in tf.items():
        if token in idf:
            vector[token] = tf_val * idf[token]
    return vector

student_vec = get_tfidf_vector(student_tokens)

matched_results = []
for idx, t in enumerate(tutors):
    # Apply Hard Filters first
    # 1. Budget filter (if specified, tutor price must be <= budget)
    if budget is not None and t['price_per_hour'] > budget:
        continue

    # 2. Subject filter (if student specified a main subject, check if it matches tutor's
    # subjects list)
    if subject_filter:
        sub_filt = subject_filter.strip().lower()
        tutor_subs = [s.strip().lower() for s in t['subjects'].split(',')]
        # If subject filter is not in tutor subjects list, skip or lower score
        # Let's check for substring match
        matched_sub = False
        for s in tutor_subs:
            if sub_filt in s or s in sub_filt:
                matched_sub = True
                break
        if not matched_sub:
            continue # Hard filter out if subject doesn't match at all

    # Calculate TF-IDF Cosine Similarity
    tutor_vec = get_tfidf_vector(tutor_docs[idx])
    sim_score = calculate_cosine_similarity(student_vec, tutor_vec)

    # Boost score slightly if subject matches directly
    if subject_filter:
        sub_filt = subject_filter.strip().lower()
        if sub_filt in t['subjects'].lower():
            sim_score += 0.1 # Boost score

    # Cap matching percentage
    match_percentage = min(100.0, max(0.0, sim_score * 100))

    # If there's no similarity score (e.g. student query consists of stop words),
    # but the tutor matched hard filters, we give a baseline score (e.g. 50%)
    if match_percentage == 0.0:
        match_percentage = 50.0

    # Round score
    match_percentage = round(match_percentage, 1)

    # Append result
    t_result = t.copy()

```

```
t_result['match_score'] = match_percentage
matched_results.append(t_result)

# Sort results by match_score descending
matched_results.sort(key=lambda x: x['match_score'], reverse=True)
return matched_results
```

ไฟล์ที่เขียน: main.py

ตำแหน่งไฟล์ในโปรเจกต์: main.py

รายละเอียดการทำงาน:

ไฟล์ระบบ API หลักของแอปพลิเคชันหลังบ้านที่ถูกสร้างขึ้นด้วย FastAPI ทำหน้าที่กำหนด API Route ทั้งหมดเพื่อทำงานร่วมกับหน้าจอมือถือ (LINE LIFF), สร้างฐานข้อมูลอัตโนมัติในการรันครั้งแรก, ถอดรหัสรูปภาพ Base64 แปลงเป็นภาพไบนารีส่งไปประมวลผลที่ AI Engine รวมถึงจัดเก็บและเรียกค้นนัดหมายเรียน และให้บริการไฟล์ Static หน้ากากเว็บของ LIFF Simulator

```
import json
import base64
import os
from fastapi import FastAPI, Depends, HTTPException, status
from fastapi.staticfiles import StaticFiles
from fastapi.responses import RedirectResponse
from sqlalchemy.orm import Session
from typing import List

import models
import schemas
from database import engine, get_db
from ai_engine import verify_faces, extract_face_landmarks, match_tutors

# Create Database tables
models.Base.metadata.create_all(bind=engine)

app = FastAPI(title="TutorMatch AI Portal", description="Tutor Matching and Face Verification System")

# Helper to decode base64 string
def decode_base64_image(base64_str: str) -> bytes:
    try:
        if "," in base64_str:
            base64_str = base64_str.split(",")[1]
        return base64.b64decode(base64_str)
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail=f"รูปแบบข้อมูลรูปภาพไม่ถูกต้อง: {str(e)}"
        )

# API: Register User
@app.post("/api/register", response_model=schemas.UserResponse)
def register_user(user: schemas.UserCreate, db: Session = Depends(get_db)):
    db_user = db.query(models.User).filter(models.User.username == user.username).first()
    if db_user:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="ชื่อผู้ใช้งานนี้ถูกใช้ไปแล้ว"
        )

    new_user = models.User(
        username=user.username,
        name=user.name,
```

```

        role=user.role
    )
    db.add(new_user)
    db.commit()
    db.refresh(new_user)
    return new_user

# API: Register Biometrics (Face)
@app.post("/api/biometrics/register")
def register_face(req: schemas.FaceRegisterRequest, db: Session = Depends(get_db)):
    user = db.query(models.User).filter(models.User.username == req.username).first()
    if not user:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="ไม่พบผู้ใช้งานในระบบ"
        )

    image_bytes = decode_base64_image(req.image_base64)
    landmarks = extract_face_landmarks(image_bytes)

    if landmarks is None:
        raise HTTPException(
            status_code=status.HTTP_422_UNPROCESSABLE_ENTITY,
            detail="ไม่สามารถตรวจจับใบหน้าในรูปภาพได้ กรุณาลoadภาพหน้าตรงให้ชัดเจน"
        )

    # Save face landmarks as JSON text or face image (now base64 string from LBPH extract
    )
    landmarks_json = landmarks # In LBPH it is a base64 string

    # Check if biometric profile already exists
    biometric = db.query(models.BiometricProfile).filter(models.BiometricProfile.user_id
    == user.id).first()
    if biometric:
        biometric.face_landmarks = landmarks_json
    else:
        biometric = models.BiometricProfile(user_id=user.id, face_landmarks=landmarks_json)
        db.add(biometric)

    db.commit()
    return {"status": "success", "message": "ลงทะเบียนข้อมูลสแกนใบหน้าสำเร็จแล้ว"}

# API: Verify Face (Login/Auth)
@app.post("/api/biometrics/verify")
def verify_face(req: schemas.FaceVerifyRequest, db: Session = Depends(get_db)):
    user = db.query(models.User).filter(models.User.username == req.username).first()
    if not user:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="ไม่พบผู้ใช้งานในระบบ"
        )

    biometric = db.query(models.BiometricProfile).filter(models.BiometricProfile.user_id
    == user.id).first()
    if not biometric:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="ผู้ใช้นี้ยังไม่ได้ลงทะเบียนข้อมูลใบหน้าเข้าระบบ"
        )

    image_bytes = decode_base64_image(req.image_base64)

```

```

# Verify landmarks (LBPH)
# Threshold for LBPH is typically 75.0 (lower is more strict)
verified, distance, error_msg = verify_faces(biometric.face_landmarks, image_bytes, threshold=75.0)

if error_msg:
    raise HTTPException(
        status_code=status.HTTP_422_UNPROCESSABLE_ENTITY,
        detail=error_msg
    )

if not verified:
    raise HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail=f"การยืนยันใบหน้าไม่สำเร็จ (Distance: {distance:.2f}) กรุณาลองใหม่อีกครั้ง"
    )

return {
    "status": "success",
    "message": "ยืนยันตัวตนสำเร็จ",
    "distance": distance,
    "user": {
        "id": user.id,
        "username": user.username,
        "name": user.name,
        "role": user.role
    }
}

# API: Save Tutor Profile
@app.post("/api/tutor/profile", response_model=schemas.TutorProfileResponse)
def save_tutor_profile(
    profile: schemas.TutorProfileCreate,
    username: str, # Sent via query parameters for simulation
    db: Session = Depends(get_db)
):
    user = db.query(models.User).filter(models.User.username == username).first()
    if not user:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="ไม่พบผู้ใช้งานในระบบ"
        )

    if user.role != "tutor":
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="บัญชีผู้ใช้นี้ไม่ใช่บัญชีผู้สอน (Tutor)"
        )

    db_profile = db.query(models.TutorProfile).filter(models.TutorProfile.user_id == user.id).first()

    # Prep combined text vector index
    vector_text = f"{profile.subjects} {profile.expertise} {profile.teaching_style} {profile.education} {profile.availability}"

    if db_profile:
        db_profile.education = profile.education
        db_profile.subjects = profile.subjects
        db_profile.expertise = profile.expertise

```

```

db_profile.teaching_style = profile.teaching_style
db_profile.price_per_hour = profile.price_per_hour
db_profile.availability = profile.availability
db_profile.contact_line = profile.contact_line
db_profile.contact_phone = profile.contact_phone
db_profile.contact_email = profile.contact_email
db_profile.appointment_instruction = profile.appointment_instruction
db_profile.profile_vector_text = vector_text
else:
    db_profile = models.TutorProfile(
        user_id=user.id,
        education=profile.education,
        subjects=profile.subjects,
        expertise=profile.expertise,
        teaching_style=profile.teaching_style,
        price_per_hour=profile.price_per_hour,
        availability=profile.availability,
        contact_line=profile.contact_line,
        contact_phone=profile.contact_phone,
        contact_email=profile.contact_email,
        appointment_instruction=profile.appointment_instruction,
        profile_vector_text=vector_text
    )
    db.add(db_profile)

db.commit()
db.refresh(db_profile)
return db_profile

# API: Get List of all Tutors
@app.get("/api/tutors")
def get_all_tutors(db: Session = Depends(get_db)):
    tutor_profiles = db.query(models.TutorProfile).all()
    results = []
    for p in tutor_profiles:
        results.append({
            "user_id": p.user_id,
            "name": p.user.name,
            "education": p.education,
            "subjects": p.subjects,
            "expertise": p.expertise,
            "teaching_style": p.teaching_style,
            "price_per_hour": p.price_per_hour,
            "availability": p.availability,
            "contact_line": p.contact_line or "-",
            "contact_phone": p.contact_phone or "-",
            "contact_email": p.contact_email or "-",
            "appointment_instruction": p.appointment_instruction or "ติดต่อผ่านช่องทางการติดต่อหลัก"
        })
    return results

# API: Search & Match Tutors (AI Matching)
@app.post("/api/student/match", response_model=List[schemas.MatchedTutorResponse])
def search_and_match_tutors(req: schemas.StudentMatchRequest, db: Session = Depends(get_db)):
    student = db.query(models.User).filter(models.User.username == req.username).first()
    if not student:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="ไม่พบข้อมูลผู้เรียนในระบบ"
        )

```

```

    )

    # Get all tutors in db
    tutor_profiles = db.query(models.TutorProfile).all()
    if not tutor_profiles:
        return []

    tutor_list = []
    for p in tutor_profiles:
        tutor_list.append({
            "user_id": p.user_id,
            "name": p.user.name,
            "education": p.education,
            "subjects": p.subjects,
            "expertise": p.expertise,
            "teaching_style": p.teaching_style,
            "price_per_hour": p.price_per_hour,
            "availability": p.availability,
            "contact_line": p.contact_line or "-",
            "contact_phone": p.contact_phone or "-",
            "contact_email": p.contact_email or "-",
            "appointment_instruction": p.appointment_instruction or "ติดต่อผู้สอนโดยตรง",
            "profile_text": p.profile_vector_text or ""
        })

    # Compute Match Score using NLP Similarity
    matched_results = match_tutors(
        student_req=req.requirements_text,
        tutors=tutor_list,
        budget=req.budget,
        subject_filter=req.subject_filter
    )

    # Save the highest match history (if results found)
    if matched_results:
        best_match = matched_results[0]
        history = models.MatchHistory(
            student_id=student.id,
            requirements_text=req.requirements_text,
            matched_tutor_id=best_match[0];user_id,
            score=best_match[0];match_score
        )
        db.add(history)
        db.commit()

    return matched_results

# API: Create Lesson Appointment
@app.post("/api/appointments", response_model=schemas.AppointmentResponse)
def create_appointment(req: schemas.AppointmentCreate, db: Session = Depends(get_db))
:
    student = db.query(models.User).filter(models.User.username == req.student_username).
        first()
    if not student:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="ไม่พบข้อมูลผู้เรียนในระบบ"
        )

    tutor = db.query(models.User).filter(models.User.id == req.tutor_id).first()
    if not tutor:

```



```

    raise HTTPException(
        status_code=status.HTTP_404_NOT_FOUND,
        detail="ไม่พบข้อมูลตัวเตอรในระบบ"
    )

appointment = models.Appointment(
    student_id=student.id,
    tutor_id=tutor.id,
    appointment_date=req.appointment_date,
    appointment_time=req.appointment_time,
    notes=req.notes
)

db.add(appointment)
db.commit()
db.refresh(appointment)

# Map to schema response
return schemas.AppointmentResponse(
    id=appointment.id,
    student_id=appointment.student_id,
    student_name=student.name,
    tutor_id=appointment.tutor_id,
    tutor_name=tutor.name,
    appointment_date=appointment.appointment_date,
    appointment_time=appointment.appointment_time,
    notes=appointment.notes,
    status=appointment.status,
    created_at=appointment.created_at
)

# API: List Appointments for a User
@app.get("/api/appointments", response_model=List[schemas.AppointmentResponse])
def get_appointments(username: str, db: Session = Depends(get_db)):
    user = db.query(models.User).filter(models.User.username == username).first()
    if not user:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="ไม่พบข้อมูลผู้ใช้งานในระบบ"
        )

    if user.role == "tutor":
        appointments = db.query(models.Appointment).filter(models.Appointment.tutor_id == user.id).all()
    else:
        appointments = db.query(models.Appointment).filter(models.Appointment.student_id == user.id).all()

    results = []
    for appt in appointments:
        results.append(schemas.AppointmentResponse(
            id=appt.id,
            student_id=appt.student_id,
            student_name=appt.student.name,
            tutor_id=appt.tutor_id,
            tutor_name=appt.tutor.name,
            appointment_date=appt.appointment_date,
            appointment_time=appt.appointment_time,
            notes=appt.notes,
            status=appt.status,
            created_at=appt.created_at

```

```
    ))
    return results

# Redirect Root to LIFF Simulator
@app.get("/")
def read_root():
    return RedirectResponse(url="/static/index.html")

# Create static directory if it doesn't exist
os.makedirs("static", exist_ok=True)

# Mount static files
app.mount("/static", StaticFiles(directory="static"), name="static")
```

ไฟล์ที่เขียน: static/index.html

ตำแหน่งไฟล์ในโปรเจกต์: static/index.html

รายละเอียดการทำงาน:

หน้าจอบริบทหน้าบ้าน (Frontend UI) ที่ถูกพัฒนาแบบ Single Page Application

เพื่อจำลองพฤติกรรมบนแอปพลิเคชัน LINE LIFF ที่ใช้โทนสีสว่างด้วย Vanilla CSS

และมี JavaScript จัดการตรรกะฝั่งผู้ใช้:

- เรียกเปิดกล้องหน้ามือถือ/คอมพิวเตอร์ผ่านเบราว์เซอร์ด้วย `mediaDevices.getUserMedia`

และถ่ายรูปรูผ่าน Canvas

- มีฟังก์ชันจำลองสร้างข้อมูลตัวต่อ 5 สาขาวิชาอัตโนมัติ เพื่อให้ผู้ใช้ทดสอบ Matching ได้ง่ายขึ้น

- แสดงผลคะแนน AI Match เปอร์เซ็นต์ พร้อมระบุ LINE ID

และปุ่มกดจองเวลานัดหมายเรียนแบบ Interactive

```
<!DOCTYPE html>
<html lang="th">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>TutorMatch AI - Portal & LIFF Simulator</title>
  <!-- Google Fonts -->
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&family=Noto+Sans+Thai:wght@300;400;500;600;700&display=swap" rel="stylesheet">

  <style>
    :root {
      --bg-color: #f8fafc;
      --card-bg: rgba(255, 255, 255, 0.9);
      --card-border: rgba(226, 232, 240, 0.9);
      --text-primary: #0f172a;
      --text-secondary: #475569;
      --primary: #7c3aed; /* Violet */
      --primary-light: #f5f3ff;
      --secondary: #059669; /* Emerald */
      --secondary-light: #ecfdf5;
      --accent: #2563eb; /* Blue */
      --warning: #d97706;
      --danger: #dc2626;
      --line-green: #06c755;

      --gradient-1: linear-gradient(135deg, #4f46e5 0%, #7c3aed 50%, #db2777 100%);
      --shadow-sm: 0 1px 3px rgba(0,0,0,0.05);
      --shadow-md: 0 10px 25px -3px rgba(15, 23, 42, 0.05), 0 4px 6px -2px rgba(15, 23, 42, 0.05);
      --shadow-lg: 0 20px 50px -12px rgba(15, 23, 42, 0.1);
      --radius-lg: 16px;
      --radius-md: 10px;
    }

    * {
      box-sizing: border-box;
      margin: 0;
      padding: 0;
```

```
}

body {
  background-color: var(--bg-color);
  background-image:
    radial-gradient(at 0% 0%, rgba(99, 102, 241, 0.08) 0px, transparent 50%),
    radial-gradient(at 100% 0%, rgba(236, 72, 153, 0.08) 0px, transparent 50%),
    radial-gradient(at 50% 100%, rgba(139, 92, 246, 0.05) 0px, transparent 50%);
  background-attachment: fixed;
  color: var(--text-primary);
  font-family: 'Inter', 'Noto Sans Thai', sans-serif;
  min-height: 100vh;
  display: flex;
  flex-direction: column;
  padding-bottom: 80px;
}

.container {
  max-width: 1100px;
  margin: 0 auto;
  padding: 20px;
  width: 100%;
}

/* Navbar */
nav {
  background: rgba(255, 255, 255, 0.8);
  backdrop-filter: blur(12px);
  border-bottom: 1px solid var(--card-border);
  padding: 15px 30px;
  display: flex;
  justify-content: space-between;
  align-items: center;
  position: sticky;
  top: 0;
  z-index: 100;
  box-shadow: var(--shadow-sm);
}

.logo-area {
  display: flex;
  align-items: center;
  gap: 10px;
  font-weight: 700;
  font-size: 1.3rem;
  background: var(--gradient-1);
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
}

.nav-status {
  display: flex;
  align-items: center;
  gap: 12px;
  font-size: 0.9rem;
}

.status-badge {
  background: rgba(15, 23, 42, 0.05);
  padding: 6px 14px;
  border-radius: 20px;
```

```

    font-weight: 500;
    border: 1px solid rgba(15, 23, 42, 0.05);
    transition: all 0.3s;
  }

  .status-badge.active-user {
    background: var(--primary-light);
    border-color: rgba(124, 58, 237, 0.2);
    color: var(--primary);
  }

  /* Buttons */
  .btn {
    padding: 10px 20px;
    border-radius: var(--radius-md);
    font-size: 0.92rem;
    font-weight: 500;
    cursor: pointer;
    border: none;
    transition: all 0.3s cubic-bezier(0.4, 0, 0.2, 1);
  }

  ... [โค้ดส่วนการแสดงผลและสไลด์ซีทเพิ่มเติม ละไว้ในเอกสารเพื่อความกระชับ] ...

  data.forEach(appt => {
    const statusClass = appt.status === 'approved' ? 'status-approved' : appt.status ===
      'declined' ? 'status-declined' : '';
    const statusText = appt.status === 'pending' ? 'รออนุมัติ' : appt.status === 'approve
      d' ? 'อนุมัติแล้ว' : 'ปฏิเสธ';
    const statusBadgeClass = appt.status === 'pending' ? 'status-pending-badge' : 'status
      -approved-badge';

    const oppRole = currentUser.role === 'student' ? 'ติวเตอร์' : 'นักเรียน';
    const oppName = currentUser.role === 'student' ? appt.tutor_name : appt.student_name;

    container.innerHTML += `
      <div class="appt-card ${statusClass}">
        <div class="appt-header">
          <span> ${oppRole}: ${oppName}</span>
          <span class="appt-status-badge ${statusBadgeClass}">${statusText}</span>
        </div>
        <div class="appt-details"> <b>วันที่:</b> ${appt.appointment_date}</div>
        <div class="appt-details"> <b>เวลา:</b> ${appt.appointment_time}</div>
        ${appt.notes ? `<div class="appt-details" style="margin-top:5px; font-style:italic;">
          ${appt.notes}</div>` : ''}
      </div>
    `;
  });
} else {
  container.innerHTML = `<p style="color: var(--text-secondary); font-style: italic; fo
    nt-size: 0.85rem;">ยังไม่มีรายการนัดหมายจองเรียน</p>`;
}
} catch (e) {
  console.error("Error loading appointments", e);
}
}

// --- SESSION MANAGEMENT ---
function updateNavbarStatus() {
  const statusText = document.getElementById('user-status-text');
  const logoutBtn = document.getElementById('logout-btn');
  const apptsCard = document.getElementById('my-appointments-card');

```

```

if (currentUser) {
  statusText.innerText = `ลงชื่อเข้าใช้: ${currentUser.name} (${currentUser.role ===
    'tutor' ? 'ติวเตอร์' : 'ผู้เรียน'})`;
  statusText.classList.add('active-user');
  logoutBtn.classList.remove('hidden');
  apptsCard.classList.remove('hidden');
} else {
  statusText.innerText = 'ไม่ได้ลงชื่อเข้าใช้งาน';
  statusText.classList.remove('active-user');
  logoutBtn.classList.add('hidden');
  apptsCard.classList.add('hidden');
}
}

function showRolePanel() {
  const defaultMsg = document.getElementById('portal-default-msg');
  const tutorView = document.getElementById('portal-tutor-view');
  const studentView = document.getElementById('portal-student-view');

  defaultMsg.classList.add('hidden');

  if (currentUser.role === 'tutor') {
    tutorView.classList.remove('hidden');
    studentView.classList.add('hidden');
    loadTutorProfile();
  } else {
    studentView.classList.remove('hidden');
    tutorView.classList.add('hidden');
  }
}

async function loadTutorProfile() {
  try {
    const response = await fetch('/api/tutors');
    const tutors = await response.json();
    const profile = tutors.find(t => t.user_id === currentUser.id);
    if (profile) {
      document.getElementById('tutor-edu').value = profile.education;
      document.getElementById('tutor-subjects').value = profile.subjects;
      document.getElementById('tutor-expertise').value = profile.expertise;
      document.getElementById('tutor-style').value = profile.teaching_style;
      document.getElementById('tutor-price').value = profile.price_per_hour;
      document.getElementById('tutor-avail').value = profile.availability;
      document.getElementById('tutor-contact-line').value = profile.contact_line !== '-' ?
        profile.contact_line : "";
      document.getElementById('tutor-contact-phone').value = profile.contact_phone !== '-'
        ? profile.contact_phone : "";
      document.getElementById('tutor-contact-email').value = profile.contact_email !== '-'
        ? profile.contact_email : "";
      document.getElementById('tutor-appointment-instruction').value = profile.appointment_
        instruction !== 'ติดต่อผู้สอนโดยตรง' ? profile.appointment_instruction : "";
    }
  } catch (e) {
    console.error("Error loading tutor profile", e);
  }
}

function logout() {
  currentUser = null;
  localStorage.removeItem('currentUser');
}

```

```
updateNavbarStatus();

document.getElementById('portal-default-msg').classList.remove('hidden');
document.getElementById('portal-tutor-view').classList.add('hidden');
document.getElementById('portal-student-view').classList.add('hidden');

document.getElementById('auth-feedback').innerHTML = "";
document.getElementById('login-username').value = "";

stopWebcam();
toggleAuthTab('login');
}

function stopWebcam() {
  if (webcamStream) {
    webcamStream.getTracks().forEach(track => track.stop());
    webcamStream = null;

    const video = document.getElementById('webcam-feed');
    video.srcObject = null;
    video.classList.add('hidden');
    document.getElementById('webcam-placeholder').classList.remove('hidden');
  }
}

function showFeedback(element, message, type) {
  element.innerHTML = `<div class="alert alert-${type}">${message}</div>`;
}
</script>
</body>
</html>
```